

SIKSHA 'O' ANUSANDHAN
DEEMED TO BE UNIVERSITY

Admission Batch:

Session:

Laboratory Record

**Python for Computer Science and Data Science 2 (CSE
3651)**

Submitted by

Name: _____

Registration No.: _____

Branch: _____

Semester: _____ Section: _____



**Department of Computer Science & Engineering Faculty of
Engineering & Technology (ITER)**

Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030

INDEX

[illegible]

MINOR ASSIGNMENT-1: OBJECT-ORIENTED PROGRAMMING (OOP)

Q1. What is the significance of classes in Python programming, and how do they contribute to object oriented programming?

Ans - Classes in Python allow the creation of user-defined data structures with attributes (variables) and methods (functions) that operate on those attributes .

They enable:

- **Encapsulation:** Grouping related data and methods together.
- **Inheritance:** Reusing code from existing classes.
- **Polymorphism:** Implementing methods that work differently based on the class.
- **Code Organization:** Improving modularity and maintainability.

Q2. Create a custom Python class for managing a bank account with basic functionalities like deposit and withdrawal?

Ans -

```
#Name - Arman Singh RegdNo.- 2241019468
from decimal import Decimal
class BankAccount:
    def __init__(self,name,balance):
        self.name=name
        self.balance=balance
    def BankAccount(self,balance):
        self.balance=balance
    def deposit(self,amount):
        self.balance+=amount
        print(f'Deposited amount: {amount}\nBalance: {self.balance}')
    def withdraw(self,amount):
        self.balance-=amount
        print(f'Withdraw amount: {amount}\nBalance: {self.balance}')
    def display(self):
        print(f'Account Holder: {self.name}\nBalance: {self.balance}')
a1=BankAccount('Arman',0)
a2=BankAccount('Amit',Decimal('100.12'))
a1.display()
a1.deposit(1000)
a1.withdraw(500)
a1.display()
a2.display()
```

Output:

```
Account Holder: Arman
Balance: 0
Deposited amount: 1000
Balance: 1000
Withdraw amount: 500
Balance: 500
Account Holder: Arman
Balance: 500
Account Holder: Amit
Balance: 100.12
```

>>>

Q3. Create a Book class that contains multiple Chapters, where each Chapter has a title and page count. Write code to initialize a Book object with three chapters and display the total page count of the book.

Ans -

```
#Name - Arman Singh   RegdNo- 2241019468
class Chapter:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

class Book:
    def __init__(self, title):
        self.title = title
        self.chapters = []

    def add_chapter(self, chapter):
        self.chapters.append(chapter)

    def total_pages(self):
        return sum(chapter.pages for chapter in self.chapters)

# Example Usage
book = Book("Python Programming")
book.add_chapter(Chapter("Introduction", 20))
book.add_chapter(Chapter("OOP Concepts", 30))
book.add_chapter(Chapter("Advanced Topics", 50))

print(f"Total Pages: {book.total_pages()}")
```

Output

```
Total Pages: 100
```

Q4. How does Python enforce access control to class attributes, and what is the difference between public, protected, and private attributes?

Ans- • **Public Attributes:** Accessible from anywhere (`self.var`)

- **Protected Attributes:** Conventionally prefixed with `_var`, accessible but should not be modified directly.
- **Private Attributes:** Prefixed with `__var`, not directly accessible outside the class.

Example:

```
class Test:
    def __init__(self):
        self.public = "I am public"
        self._protected = "I am protected"
        self.__private = "I am private"
    def show(self):
        return self.__private
obj = Test()
print(obj.public) # Accessible
print(obj._protected) # Accessible, but not recommended
# print(obj.__private) # Error
print(obj.show()) # Access private via method
```

Q5. Write a Python program using a Time class to input a given time in 24-hour format and convert it to a 12-hour format with AM/PM. The program should also validate time strings to ensure they are in the correct HH:MM:SS format. Implement a method to check if the time is valid and return an appropriate message.

Ans-

```
#Name - Arman Singh RegdNo - 2241019468
import re
class Time:
    def __init__(self, time_str):
        self.time_str = time_str

    def is_valid_time(self):
        pattern = r"^([01]\d|2[0-3]):([0-5]\d):([0-5]\d)$"
        return re.match(pattern, self.time_str)

    def to_12_hour_format(self):
        if not self.is_valid_time():
            return "Invalid time format!"

        hh, mm, ss = map(int, self.time_str.split(":"))
        period = "AM" if hh < 12 else "PM"
        hh = hh % 12 or 12
        return f"{hh}:{mm:02d}:{ss:02d} {period}"

# Example Usage
time = Time("14:30:45")
print(time.to_12_hour_format())
```

Output

```
2:30:45 PM
```

Q6. Write a Python program that uses private attributes for creating a BankAccount class. Implement methods to deposit, withdraw, and display the balance, ensuring direct access to the balance attribute is restricted. Explain why using private attributes can help improve data security and prevent accidental modifications.

Ans-

```
#Name - Arman Singh   RegdNo- 2241019468
class BankAccount:
    def __init__(self, balance=0):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if amount > self.__balance:
            print("Insufficient funds!")
        else:
            self.__balance -= amount

    def get_balance(self):
        return self.__balance
# Example Usage
account = BankAccount(500)
account.deposit(200)
print(account.get_balance()) # Output: 700
|
```

Using private attributes prevents accidental modification.

Q7. Write a Python program to simulate a card game using object-oriented principles. The program should include a Card class to represent individual playing cards, a Deck class to represent a deck of cards, and a Player class to represent players receiving cards. Implement a shuffle method in the Deck class to shuffle the cards and a deal method to distribute cards to players. Display each player's hand after dealing.

Ans-


```
#Name- Arman Singh RegdNo.- 2241019466|
import random
class Card:
    def __init__(self, suit, value):
        self.suit = suit
        self.value = value
    def __str__(self):
        return f"{self.value} of {self.suit}"
class Deck:
    suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
    values = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"]

    def __init__(self):
        self.cards = [Card(suit, value) for suit in self.suits for value in self.values]
        random.shuffle(self.cards)

    def deal(self, num):
        return [self.cards.pop() for _ in range(num)]

class Player:
    def __init__(self, name):
        self.name = name
        self.hand = []

    def receive_cards(self, cards):
        self.hand.extend(cards)

    def show_hand(self):
        return ", ".join(str(card) for card in self.hand)
# Example Usage
deck = Deck()
player1 = Player("Alice")
player2 = Player("Bob")
player1.receive_cards(deck.deal(5))
player2.receive_cards(deck.deal(5))
print(f"Alice's Hand: {player1.show_hand()}")
print(f"Bob's Hand: {player2.show_hand()}")
```

Output:

```
>>> Alice's Hand: 9 of Clubs, K of Diamonds, 4 of Spades, 2 of Hearts, 7 of Spades
>>> Bob's Hand: 10 of Clubs, 10 of Diamonds, J of Hearts, 5 of Hearts, 10 of Spades
>>> |
```

Q8. Write a Python program that defines a base class Vehicle with attributes make and model, and a method display info(). Create a subclass Car that inherits from Vehicle and adds an additional attribute num doors. Instantiate both Vehicle and Car objects, call their display info() methods, and explain how the subclass inherits and extends the functionality of the base class.

Ans-

```
#Name-Arman Singh RegcNo.- 2241019468
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def display_info(self):
        return f"Vehicle: {self.make} {self.model}"

class Car(Vehicle): # Car inherits from Vehicle
    def __init__(self, make, model, num_doors):
        super().__init__(make, model) # Calling the base class constructor
        self.num_doors = num_doors

    def display_info(self): # Overriding the display_info method
        return f"Car: {self.make} {self.model}, Doors: {self.num_doors}"

# Instantiate Vehicle and Car objects
vehicle = Vehicle("Toyota", "Camry")
car = Car("Honda", "Civic", 4)

# Calling display_info() for both objects
print(vehicle.display_info()) # Output: Vehicle: Toyota Camry
print(car.display_info()) # Output: Car: Honda Civic, Doors: 4
|
```

Output

```
| Vehicle: Toyota Camry
| Car: Honda Civic, Doors: 4
|>>> |
```

- **Inheritance:**

- • The `Car` class **inherits** from the `Vehicle` class, meaning it automatically gets access to `make` and `model` attributes and the `display_info()` method.
- This prevents code duplication and improves **reusability**.

- **Extending Functionality:**

- • The `Car` class **adds** an extra attribute `num_doors` that is specific to cars.
- It **overrides** the `display_info()` method to include information about the number of doors.

Q9. Write a Python program demonstrating polymorphism by creating a base class `Shape` with a method `area()`, and two subclasses `Circle` and `Rectangle` that override the `area()` method. Instantiate objects of both subclasses and call the `area()` method. Explain how polymorphism simplifies working with different shapes in an inheritance hierarchy.

Ans-

```
#Name- Arman Singh RegdNo.-2241019468
class Shape:
    def area(self):
        raise NotImplementedError("Subclass must implement this method")
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return 3.14 * self.radius * self.radius
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height
    def area(self):
        return 0.5 * self.base * self.height
# Using polymorphism to calculate areas
shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]
for shape in shapes:
    print(f"Area: {shape.area()}")
```

Output:

```
Area: 78.5
Area: 24
Area: 12.0
>>>
```

- **Single Interface for Multiple Shapes:**

- The Shape class defines a common method `area()`, which is implemented differently in Circle, Rectangle, and Triangle.
- We can use the same method (`area()`) on different objects **without checking their types**.

- **Code Reusability and Scalability:**

- We can add new shape classes (e.g., Square, Pentagon) without modifying the existing code.
- The program remains **flexible** and **extensible**.

- **Simplifies Iteration & Function Calls:**

- We can store all shape objects in a list and iterate over them **without knowing their exact types**.
- This reduces the need for if-else or switch-case statements, making the code **cleaner and more maintainable**.

Q10. Implement the CommissionEmployee class with init , earnings, and repr methods. Include properties for personal details and sales data. Create a test script to instantiate the object, display earnings, modify sales data, and handle data validation errors for negative values.

Ans-

```
#Name- Arman Singh RegdNo- 2241019468
class CommissionEmployee:
    def __init__(self, name, sales_amount, commission_rate):
        if sales_amount < 0 or commission_rate < 0:
            raise ValueError("Sales amount and commission rate must be non-negative")

        self.name = name
        self.sales_amount = sales_amount
        self.commission_rate = commission_rate

    def earnings(self):
        return self.sales_amount * self.commission_rate

    def __repr__(self):
        return (f"CommissionEmployee(Name: {self.name}, Sales: {self.sales_amount}, "
                f"Commission Rate: {self.commission_rate}, Earnings: {self.earnings()})")

employee = CommissionEmployee("Alice", 5000, 0.10)
print(employee)
employee.sales_amount = 6000 # Updating sales
print("Updated Earnings:", employee.earnings())
```

Output:

```
CommissionEmployee(Name: Alice, Sales: 5000, Commission Rate: 0.1, Earnings: 500.0)
Updated Earnings: 600.0
>>>
```

Q11. What is duck typing in Python? Write a Python program demonstrating duck typing by creating a function describe() that accepts any object with a speak() method. Implement two classes, Dog and Robot, each with a speak() method. Pass instances of both classes to the describe() function and explain how duck typing allows the function to work without checking the object's type.

Ans- Duck typing, which the Python documentation describes as: A programming style which does not look at an object's type to determine if it has the right interface; instead, the method or attribute is simply called or used ("If it looks like a duck and quacks like a duck, it must be a duck.")

```
#Name- Arman Singh RegdNo- 2241019468
class Dog:
    def speak(self):
        return "Woof!"
class Robot:
    def speak(self):
        return "Beep Boop!"

def describe(obj):
    print(obj.speak())

# Example Usage
dog = Dog()
robot = Robot()
describe(dog)    # Output: Woof!
describe(robot)  # Output: Beep Boop!
```

Output:

```
Woof!
Beep Boop!
>>>
```

Q12. WAP to overload the + operator to perform addition of two complex numbers using a customComplex class?

Ans-

```
#Name- Arman Singh RegdNo-2241019468
class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    def __add__(self, other):
        return Complex(self.real + other.real, self.imag + other.imag)

    def __str__(self):
        return f"{self.real} + {self.imag}i"

# Example Usage
c1 = Complex(3, 4)
c2 = Complex(1, 2)
result = c1 + c2
print(result)    # Output: 4 + 6i
```

Output:

```
4 + 6i
>>>
```

Q13. WAP to create a custom exception class in Python that displays the balance and withdrawal amount when an error occurs due to insufficient funds?

Ans-

```
# Name- Arman Singh RegdNo-2241019468
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f"Insufficient funds! Balance: {balance}"
                        f", Attempted Withdrawal: {amount}")

class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount
        return self.balance

# Example Usage
try:
    account = BankAccount(500)
    account.withdraw(600)
except InsufficientFundsError as e:
    print(e) # Output: Insufficient funds! Balance: 500, Attempted Withdrawal:
```

Output

```
>>> Insufficient funds! Balance: 500, Attempted Withdrawal: 600
```

Q14. Write a Python program using the Card data class to simulate dealing 5 cards to a player from a shuffled deck of standard playing cards. The program should print the player's hand and the number of remaining cards in the deck after the deal.

Ans-


```
# Name- Arman Singh RegdNo-2241019468
import random
from dataclasses import dataclass
@dataclass
class Card:
    suit: str
    value: str

class Deck:
    suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
    values = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"]
    def __init__(self):
        self.cards = [Card(suit, value) for suit in self.suits for value in self.values]
        random.shuffle(self.cards)
    def deal(self, num):
        return [self.cards.pop() for _ in range(num)]

# Example Usage
deck = Deck()
hand = deck.deal(5)
print("Player's Hand:", hand)
print("Remaining Cards in Deck:", len(deck.cards))
```

Output

```
Player's Hand: [Card(suit='Spades', value='6'), Card(suit='Clubs', value='K'),
Card(suit='Spades', value='10'), Card(suit='Clubs', value='4'), Card(suit='Dia
monds', value='8')]
Remaining Cards in Deck: 47
>>>
```

Q15. How do Python data classes provide advantages over named tuples in terms of flexibility and functionality? Give an example using python code.

Ans-

Advantages Over Named Tuples:

1. **Supports Methods:** Can have additional functions inside the class.
2. **Default Values:** Allows setting default values for attributes.
3. **Mutability:** Unlike named tuples, data classes allow modification.

Q16. Write a Python program that demonstrates unit testing directly within a function's docstring using the doctest module. Create a function add(a, b) that returns the sum of two numbers and includes multiple test cases in its docstring. Implement a way to automatically run the tests when the script is executed.

Ans-

```
# Name- Arman Singh RegdNo-2241019468
```

```
def add(a, b):  
    """  
    Returns the sum of two numbers.  
  
    >>> add(2, 3)  
    5  
    >>> add(-1, 1)  
    0  
    >>> add(0, 0)  
    0  
    """  
    return a + b  
  
if __name__ == "__main__":  
    import doctest  
    doctest.testmod()
```

17. Scope Resolution: object's namespace → class namespace → global namespace → built-in namespace.

```
species = Global Species  
  
class Animal:  
    species = Class Species  
  
    def __init__(self, species):  
        self.species = species  
  
    def display_species(self):  
        print("Instance species:", self.species)  
        print("Class species:", Animal.species)  
        print("Global species:", globals()['species'])  
  
a = Animal("Instance Species")  
a.display_species()
```

What will be the output when the above program is executed? Explain the scope resolution process step by step.

Ans-

Output:

Instance species: Instance Species

Class species: Class Species

Global species: Global Species

Scope Resolution Order:

1. **Instance Attribute** (self.species)
2. **Class Attribute** (Animal.species)
3. **Global Variable** (species)

Q18. Write a Python program using a lambda function to convert temperatures from Celsius to Kelvin, store the data in a tabular format using pandas, and visualize the data using a plot.

Ans-

```
# Name- Arman Singh RegdNo-2241019468

import pandas as pd
import matplotlib.pyplot as plt

# Lambda function to convert Celsius to Kelvin
c_to_k = lambda c: c + 273.15

# Sample temperature data
celsius_values = [0, 10, 20, 30, 40, 50]
kelvin_values = list(map(c_to_k, celsius_values))
# Create DataFrame
df = pd.DataFrame({'Celsius': celsius_values, 'Kelvin': kelvin_values})
# Display Table
print(df)
# Plot data
plt.plot(df['Celsius'], df['Kelvin'], marker='o', linestyle='-')
plt.xlabel('Celsius')
plt.ylabel('Kelvin')
plt.title('Celsius to Kelvin Conversion')
plt.grid()
plt.show()
```

Output



